

# Statistique descriptive

Chapitre 15 : Mise en œuvre sur Python

**Exercices corrigés**

---

Jaouad Madkour

Faculté d'Économie et de Gestion de Tanger

Statistique descriptive

## Table des matières

|                                       |   |
|---------------------------------------|---|
| <b>Partie 1 --- Énoncés</b> .....     | 1 |
| <b>Partie 2 --- Corrections</b> ..... | 3 |

## 1 Exercices du chapitre

### Exercice 1 — numpy contre pandas : deux conventions opposées

On reprend les temps de trajet (en minutes) de 8 employés du chapitre 7 : 15, 22, 18, 30, 25, 20, 28, 24, de variance **descriptive**  $V(x) = 22,1875$ .

- (a) Sachant que numpy utilise par défaut  $\text{ddof}=0$  (division par  $n$ ), que renverrait `np.var(x)` ?
- (b) Sachant que pandas utilise par défaut  $\text{ddof}=1$  (division par  $n - 1$ ), que renverrait `pd.Series(x).var()` ?
- (c) Un étudiant convertit son tableau numpy en `pd.Series` avant de calculer la variance, et s'étonne que le résultat ne corresponde plus à son calcul manuel. Expliquer précisément l'origine de cet écart.

### Exercice 2 — L'argument fisher de scipy.stats.kurtosis

On reprend les notes du chapitre 8 : 60, 62, 58, 61, 59, 95, 57, pour lesquelles  $\gamma_2 \approx 5,02$  (convention du cours, référence 3).

- (a) Calculer l'excès d'aplatissement  $\gamma_2 - 3$ .
- (b) Sachant que `scipy.stats.kurtosis()` renvoie par défaut l'excès (`fisher=True`), que faut-il préciser pour obtenir directement  $\gamma_2 \approx 5,02$  ?
- (c) Un rapport indique « aplatissement = 2,02 » sans préciser la convention utilisée. Ce chiffre est-il, selon la convention, synonyme d'une distribution **leptokurtique** ou **platykurtique** ? Justifier dans les deux cas de figure possibles.

### Exercice 3 — pd.cut et la borne right

On reprend les classes de revenu des chapitres 3 à 9 : bornes 0, 10, 20, 40, 80 (milliers de dh), convention `[a ; b[`.

- (a) Un ménage a un revenu de **exactement** 40. Dans quelle classe tombe-t-il selon la convention `[a ; b[` du cours ?
- (b) Avec `pd.cut(revenu, bins=[0,10,20,40,80])` (sans préciser `right=False`), quelle classe pandas attribuerait-il à ce même ménage ?
- (c) Quelle instruction complète faut-il écrire pour obtenir la convention du cours ? Ce piège est-il propre à Python, ou l'avez-vous déjà rencontré ailleurs dans ce cours ?

## 2 Exercices cumulatifs (chapitres 3 à 15)

### Exercice 4 — Bilan final : reconstruire et vérifier le revenu des chapitres 3–14

On reconstruit, à partir des classes de revenu des chapitres 3 à 14 (`[0,10[` : 200 ménages de centre 5 ; `[10,20[` : 150 de centre 15 ; `[20,40[` : 100 de centre 30 ; `[40,80[` : 50 de centre 60), un tableau `x` de 500 valeurs en répétant chaque centre de classe autant de fois que son effectif (`x = np.repeat(centres, effectifs)`).

- (a) Que renverrait `x.mean()` ? Comparer à la moyenne  $\bar{x} = 18,5$  établie au chapitre 5.
- (b) Que renverrait `x.var()` (tableau numpy,  $\text{ddof}=0$  par défaut) ? Comparer à  $V(x) = 275,25$  établi au chapitre 7.
- (c) Un collègue calcule `pd.Series(x).var()` sur ce même tableau et obtient un résultat légèrement différent. Calculer cette valeur, et expliquer pourquoi elle diffère de celle de la question (b) alors que les données sont rigoureusement identiques.

### 3 Applications en sciences économiques

#### Exercice 5 — Harmoniser les conventions dans une équipe de recherche

Une équipe de recherche en économie du travail prépare un rapport commun. L'analyste A utilise R et `sd()`, l'analyste B utilise pandas et `.std()`, l'analyste C utilise numpy et `np.std()`, chacun sur le **même** fichier de 2 000 salaires.

- (a) Ces trois analystes obtiendront-ils, sans précaution particulière, exactement le même écart-type ? Justifier à partir des conventions par défaut de chaque outil.
- (b) Sur un échantillon de 2 000 observations, l'écart relatif entre les conventions  $n$  et  $n - 1$  est-il susceptible d'être visible à l'œil nu dans un rapport arrondi à l'unité, ou reste-t-il négligeable ? Est-ce toujours le cas pour un sous-groupe de 8 observations dans une analyse détaillée par région ?
- (c) Proposer une règle de gestion de projet (une phrase) que cette équipe devrait adopter pour garantir que son rapport final soit reproductible, quel que soit le logiciel utilisé par chaque analyste.

#### Exercice 6 — L'ambiguïté de `fisher` dans un rapport de risque financier

On reprend les rendements journaliers du chapitre 8 (exercice 5) : 0,5, -0,3, 0,2, -0,1, 3,0, -2,8, 0,4 (en %), pour lesquels  $\gamma_2 \approx 3,32$  (convention du cours).

- (a) Un analyste utilise `scipy.stats.kurtosis(rendements)` sans argument et obtient 0,32. Vérifier que ce chiffre correspond bien à l'excès d'aplatissement.
- (b) L'analyste écrit dans son rapport : « aplatissement = 0,32, proche de zéro, la distribution est quasiment normale ». Un collègue, habitué à la convention  $\gamma_2$  (référence 3), pourrait-il mal interpréter cette phrase si on lui rapportait seulement le chiffre 0,32 sans préciser la convention ?
- (c) Pourquoi une confusion sur ce seul chiffre pourrait-elle avoir des conséquences concrètes dans un contexte de gestion des risques de marché (faire le lien avec la mise en garde du chapitre 8 sur la Value at Risk gaussienne) ? Quelle bonne pratique rédactionnelle élimine totalement ce risque de confusion ?

## 1 Exercices du chapitre

### Correction — Exercice 1

- (a) `np.var(x)` divise par  $n = 8$  : il renverrait **exactement** 22,1875, la variance descriptive du chapitre 7.
- (b) `pd.Series(x).var()` divise par  $n - 1 = 7$  :  $\sum (x_i - \bar{x})^2 = 8 \times 22,1875 = 177,5$ , donc le résultat serait  $177,5/7 \approx 25,357$ .
- (c) L'écart vient uniquement du **changement de convention par défaut** entre les deux bibliothèques : `numpy` utilise `ddof=0` (division par  $n$ , la convention **descriptive** du cours), tandis que `pandas` utilise `ddof=1` (division par  $n - 1$ , la convention **inférentielle**). Les données n'ont pourtant pas changé d'une virgule : c'est le simple fait de convertir le tableau en `pd.Series` qui a changé la formule appliquée. La solution est d'écrire explicitement `pd.Series(x).var(ddof=0)` pour retrouver 22,1875.

`np.var(x)` (`ddof=0` par défaut) → 22,1875

↓ même donnée, deux défauts opposés

`pd.Series(x).var()` (`ddof=1` par défaut) → 25,357

### Correction — Exercice 2

- (a)  $\text{Excès} = \gamma_2 - 3 \approx 5,02 - 3 = 2,02$ .
- (b) Il faut préciser `scipy.stats.kurtosis(x, fisher=False)` pour obtenir directement  $\gamma_2 \approx 5,02$  plutôt que l'excès.
- (c) Selon la convention **excès** (référence 0, celle par défaut de `scipy`),  $2,02 > 0$  signale une distribution **leptokurtique** (queues plus épaisses que la normale) — c'est la bonne lecture ici, puisque `kurtosis()` sans argument renvoie l'excès par défaut. Mais un lecteur habitué à la convention  $\gamma_2$  (référence 3) et ignorant qu'il s'agit d'un excès pourrait, à tort, lire  $2,02 < 3$  et conclure à une distribution **platykurtique** (moins pointue que la normale) — une lecture **opposée** à la réalité. C'est exactement l'avertissement du chapitre 8 et du chapitre 14 : un coefficient d'aplatissement ne s'interprète jamais sans connaître sa convention.

### Correction — Exercice 3

- (a) Selon  $[a; b[$ , un revenu de 40 tombe dans la classe  $[40, 80[$  (borne inférieure incluse).
- (b) Avec la convention par défaut de `pd.cut` (`right=True`, donc  $]a; b]$ ), un revenu de 40 tomberait dans la classe **précédente**,  $]20, 40]$  (borne supérieure incluse).
- (c) Il faut écrire `pd.cut(revenu, bins=[0,10,20,40,80], right=False)`. **Non**, ce piège n'est **pas** propre à Python : c'est exactement le même problème rencontré au chapitre 14 avec la fonction `cut()` de R, qui partage la même convention par défaut  $]a; b]$  et se corrige de la même façon avec `right = FALSE`. Deux langages différents, syntaxe presque identique, même piège.

## 2 Exercices cumulatifs (chapitres 3 à 15)

### Correction — Exercice 4

- (a) `x.mean()` renverrait  $\frac{200 \times 5 + 150 \times 15 + 100 \times 30 + 50 \times 60}{500} = \frac{9\,250}{500} = 18,5$  — exactement la moyenne du chapitre 5.
- (b) `x.var()` sur un tableau `numpy` (`ddof=0` par défaut) renverrait **exactement** 275,25, la variance descriptive du chapitre 7, puisque `numpy` divise par  $n = 500$  par défaut, en accord avec la convention du cours.
- (c) `pd.Series(x).var()` divise par  $n - 1 = 499$  :  $\sum (x_i - \bar{x})^2 = 500 \times 275,25 = 137\,625$ , donc le résultat

serait  $137\,625/499 \approx 275,802$ . La différence, bien que faible en valeur relative (0,2 % environ, comme dans l'exemple du cours sur les 500 salariés), vient du **même** mécanisme qu'à l'exercice 1 : seule la conversion en `pd.Series` change la convention de calcul (`ddof=1` au lieu de `ddof=0`), les données étant rigoureusement identiques dans les deux cas.

### 3 Applications en sciences économiques

#### Correction — Exercice 5

(a) **Non**, pas sans précaution : `sd()` en R et `.std()` en pandas divisent tous deux par  $n - 1$  par défaut, mais `np.std()` divise par  $n$  par défaut. Les analystes A et B obtiendront donc la **même** valeur entre eux, tandis que l'analyste C obtiendra une valeur légèrement **différente**, sauf à préciser explicitement `ddof=1` dans son appel à `numpy`.

(b) Sur 2 000 observations, l'écart relatif entre les conventions  $n$  et  $n - 1$  est de l'ordre de  $1/(2n) \approx 0,025\%$  : totalement **négligeable** et invisible dans un rapport arrondi à l'unité. En revanche, pour un sous-groupe de seulement 8 observations (comme dans une analyse détaillée par petite région), l'écart relatif  $1/(2 \times 8) \approx 6,25\%$  devient potentiellement **visible**, et pourrait faire diverger deux analyses menées sur le même sous-groupe avec des outils différents.

(c) Une règle de gestion de projet raisonnable serait : « préciser explicitement et documenter, dans le code et dans le rapport, la convention de degrés de liberté (`ddof`) utilisée pour **chaque** calcul de variance ou d'écart-type, quel que soit l'outil employé, plutôt que de se fier aux valeurs par défaut de chaque bibliothèque » — cette seule discipline élimine la quasi-totalité des écarts inexpliqués entre collaborateurs utilisant des outils différents.

#### Correction — Exercice 6

(a)  $\gamma_2 \approx 3,32$  (convention du cours) donne un excès  $\gamma_2 - 3 \approx 0,32$  ✓, ce qui correspond bien au chiffre obtenu par défaut avec `scipy.stats.kurtosis()` (qui renvoie l'excès, `fisher=True` par défaut).

(b) **Oui**, absolument : un collègue habitué à la convention  $\gamma_2$  (référence 3), s'il voit seulement le chiffre « 0,32 » sans savoir qu'il s'agit d'un **excès**, pourrait le comparer à tort au seuil 3 et conclure à une distribution **extrêmement** platykurtique (bien en-dessous de 3), alors qu'en réalité — dans la bonne convention — cette distribution est **légèrement leptokurtique** ( $\gamma_2 \approx 3,32 > 3$ ), donc à queues plus épaisses que la normale.

(c) Cette confusion aurait des conséquences **concrètes** en gestion des risques : comme le rappelle le chapitre 8, un  $\gamma_2$  supérieur à 3 (ou un excès positif) signale que les pertes extrêmes sont **plus fréquentes** que ne le prédit un modèle gaussien — c'est précisément l'un des mécanismes à l'origine des défaillances de la Value at Risk gaussienne lors de la crise de 2008. Un analyste convaincu à tort que ses rendements sont « quasiment normaux, voire moins risqués que la normale » (mauvaise lecture de l'excès) pourrait **sous-calibrer** une réserve de capital ou une limite de risque, avec des conséquences potentiellement sérieuses en cas de choc de marché. La bonne pratique rédactionnelle qui élimine ce risque : **toujours accompagner tout coefficient d'aplatissement de sa convention explicite dans le texte même du rapport** (par exemple « excès d'aplatissement (référence 0) = 0,32 » ou «  $\gamma_2$  (référence 3) = 3,32 »), sans jamais se contenter du mot « aplatissement » suivi d'un simple nombre.